

FORMULATION AND OPTIMIZATION OF UAV SUBSURFACE IMAGING USING A POMDP

Duncan Byrne
University of Colorado Boulder
Boulder, CO. United States
duncan.byrne@colorado.edu

Jonathan Williams
University of Colorado Boulder
Boulder, CO. United States
jonathan.williams@colorado.edu

Abstract—Different remote sensing methods for imaging subsurface targets continue to be of interest for scientists—specifically, in Earth sciences involving resource monitoring or object detection. In this paper, we propose an approach to navigating an Unmanned Aerial Vehicle (UAV) hosting a ground-penetrating radar instrument to detect subsurface targets by formulating the scenario as a multi-objective optimization problem under uncertainty. We first formulate the problem as a Markov Decision Process (MDP) where the target locations and conductive regions are known by the agent. Using Q-learning, we find the optimal policy and explore a one-dimensional cell world with up to 20 targets and conductive regions. Using our findings from Q-learning, we expand to a Partially Observable MDP (POMDP) and find an optimal policy with Soft Actor Critic (SAC). Both implementations with static and random point targets and conductive regions performed successfully with the agent imaging all targets. With the work accomplished in this paper, we hope to adapt this algorithm for future investigations for autonomous UAV platforms with both active and passive radar instrumentation. We also plan to adapt this for other remote sensing methods.

Index Terms—UAV, radar, subsurface targets, reinforcement learning.

I. INTRODUCTION

Unmanned Aerial Vehicles (UAVs) serve as a flexible remote sensing platform across many different terrestrial and planetary studies. More specifically, radar instrumentation onboard UAVs is commonly sought out by researchers and engineers due to the platform providing the most flexibility for survey design and the least amount of costs in terms of size, weight, and power (SWaP) requirements [1]. Compared to their aircraft- and satellite-based counterparts, UAV platforms can operate at much lower altitudes and can therefore provide higher spatial resolution measurements of a given area. UAV platforms are also not limited by the necessity to remain in constant motion but instead can hover over a survey zone or make repeated surveys over extended periods of time.

Scientists and engineers have recently found success in utilizing radar instrumentation onboard UAVs for the detection of subsurface targets. Such examples include detecting buried objects [2], exploring for the presence of resources [3], and rescue objectives [4]. Additionally, UAVs are often prioritized by scientists and engineers because they require less manual effort. However, both communities are seeking to develop platforms that can be operated without continuous input from a

remote Pilot in Command (PIC). While this would open new opportunities for exploration and further reduce the manual effort required for a given survey, the detection of subsurface targets using autonomous methods remains a significant challenge. More specifically, environments where electromagnetic signals are strongly attenuated by conductive media provide the greatest risk for the usage of autonomous methods in radar remote sensing for subsurface target detection.

Conductive regions refer to materials that attenuate radar signals significantly, often acting as barriers to signal penetration [5]. For example, clay-rich sediments or saline soils have electrical properties that make radar attenuation significantly difficult [6]. In contrast, glacial ice, such as that found in the polar regions of Earth, is a poor electrical conductor and is effectively transparent to a wide range of radar signals [5]. While the electrical properties, and therefore the expected attenuation in decibels per kilometer, are often known for any given material, the layout of these regions in any given survey area is often unknown a priori. This may result in misleading observations and conclusions in subsurface target studies using radar. In addition to issues with conductive regions and materials, sensor geometry and platform trajectory can also create additional sources of error and lead to additional conclusions that may not reflect the truth of a given experiment.

To address these issues in the field of remote sensing, we will formulate a theoretical decision framework for an autonomous UAV-based radar instrument where an agent (i.e., a UAV) must actively select both its trajectory and instrument-pointing strategy to minimize the time needed to extract all subsurface target information in a given area. We aim to formulate the problem as a Partially Observable Markov Decision Process (POMDP) model of a drone/UAV that will use radar to image subsurface targets through a heterogeneous, conductive media. To best replicate real-life applications, certain areas of the subsurface will be conductive and will return no signal. However, the UAV can still image the subsurface target under these regions if it moves away from these regions and augments the pointing direction of the radar. To best support the needs of the science community (i.e., the implementation of machine learning algorithms to reduce the workload of the remote PIC), we plan to use reinforcement learning to develop an algorithm that selects an optimal policy to capture the subsurface layer while avoiding the conductive regions

quickly. We considered a progression of three models to replicate this problem, beginning with a rudimentary baseline approach and extending towards a more comprehensive and accurate model. In this paper, we present approaches 1 and 2.

A. Approach 1

We will first model the problem as a Markov Decision Process (MDP) with a finite action space and a known subsurface conductivity map. There will be no uncertainty in either the instrument or the conductivity map. We will model the study site as a 1D cell world where the drone is rewarded for collecting information on all subsurface targets (the terminal state) and is penalized for wasting time. We will determine the optimal policy for the study site using Q-learning. To implement Q-learning, we will adapt existing Q-learning code written in Julia to be applicable for our custom MDP.

B. Approach 2

Our main approach involves altering the known conductivity map into a belief map with uncertainty, emulating a more realistic scenario. We will implement the uncertainty by expressing the problem as a Partially Observable MDP (POMDP). We will identify an optimal policy for the UAV to take through a Soft Actor Critic (SAC) algorithm that considers the entropy of the solution. The reward function will again push the UAV towards collecting information on all subsurface targets as quickly as possible. For this approach, we will implement Soft Actor Critic using the pre-existing Python StableBaselines3 implementation.

C. Approach 3

In future work, we plan to solve for the optimal solution through an implementation of Monte Carlo Tree Search (MCTS) algorithm over the belief space through some form of generative model. By implementing an online method for policy decision making, we move the problem towards the overall goal of autonomous decision making for radar instrumentation onboard UAVs.

II. PREVIOUS WORKS

Several previous works have explored the usage of POMDPs to model multi-target search algorithms onboard UAVs or other similar platforms (e.g., robots). For example, [7] introduced a general POMDP formulation for detecting multiple objects in a 3D state and action space. By using a Partially-Observable Upper Confidence bounds for Trees (POUCT) MCTS algorithm [8], [7] was able to determine where several objects were located at different heights within roughly 15 minutes.

[9] also explores a multi-object manipulation planning problem. Like our problem, this work uses imperfect sensors to estimate the state of the environment. [9] also uses the POUCT algorithm, as derived in [8], to determine an effective policy for the given problem setup.

UAV multi-target tracking has been previously formulated as a POMDP in [10], where the authors aimed to minimize the mean-square error between a UAV and several moving targets

as the UAV surveyed a given area. However, this differs from our work, as the targets do have the ability to move at each time step. Furthermore, the authors of the paper assume no false alarms or missed detections, and that the drone correctly associates each measurement with its corresponding target.

Similar to [11], our problem formulation involves a split state. [11] introduces a Mixed Observability MDP (MOMDP), where a given part of the state is fully known but another part is only partially observable. In our problem, the UAV position at any given moment is fully known, while other aspects such as target location and conductivity are unknown.

Where our work differs from previous papers is in the introduction of a reward structure based on the number of targets captured. For example, [7] incentivizes the detection of independent targets, which can lead to policies that do not prioritize full coverage of all targets within a limited time frame. In contrast, we propose a POMDP model that explicitly encourages the UAV to detect all targets efficiently.

In summary, we build on POMDP formulations for multi-target search by incorporating asymmetric observation noise, a global constraint on the number of targets, and environment-dependent sensing. As such, our problem aligns more closely with the needs of the remote sensing community than with traditional robotics applications.

III. PROBLEM FORMULATION

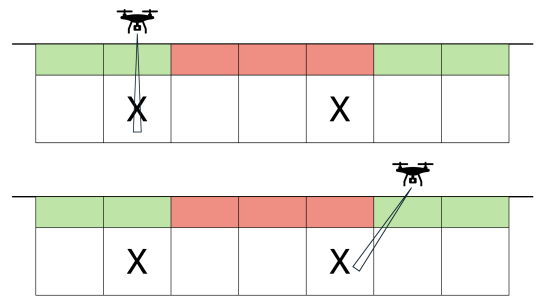


Fig. 1. Depiction of the layout of the UAV problem, showing the two different ways the UAV can scan a given subsurface target

To define the POMDP for this optimization problem, we must first consider the scope of the problem for the real world. A real-world UAV would exist in a continuous, three dimensional (x, y, z) state where it could fly at any given latitude, longitude, and altitude. Furthermore, a drone is operated in a continuous control space, allowing it to move in any direction and therefore yielding a high-dimensional (theoretically infinite) action space. This action space is further expanded by the inclusion of our radar instrument, whose steerable field of view introduces additional actions into the action space. The expanse of given states and actions leads to an infinite transition function as well. Weather additionally adds a degree of uncertainty to both the transition and observation functions.

As seen above, the true scope of the problem cannot be fully defined. As our work is an introductory point for the remote sensing field, we have simplified the problem to a one

dimensional (x), discrete state and action space with limited actions as seen in Figure 1. While not representative of the true scope of the problem, the problem formulation below and the results derived from it are still important, as they can be built upon and adapted in later works. As such, we define the POMDP as the following:

A. State Space

Similar to [11], our state will be expressed as a vector of information where part of the state is fully known and part is unknown but observable. The first element of the vector is the position p , which is a value between 1 and N . The position will be known by the drone. Additionally, the imaging status of all the targets in our survey area is encoded into the second element of the vector m , where each cell n can be encoded as a bitmask ($m \in \{0, 1\}$). This can be expressed as:

$$s \in \mathcal{S}, \quad s = (p, m)$$

The agent (i.e., the UAV) does not observe m directly, leading this to be a partially observable MDP. The problem is terminated whenever the m vector of the state is fully flipped to ones:

$$\text{terminal if } m = [1, 1, 1, \dots, 1]$$

B. Action Space

For our formulation, the drone has access to two different movement actions and three different scanning actions. The drone can move left or right. The drone can also scan in one of three directions: downward, to the left, or to the right. Finally, the drone has the ability to do nothing (hover in place).

$$\mathcal{A} = \{L, R, S_D, S_L, S_R, N\}$$

As the SAC algorithm implemented requires a continuous action space to run properly, we formulated our action space into 2 dimensions for SAC, where $\mathcal{A}_{\text{scan}}$ is the continuous scan space and $\mathcal{A}_{\text{move}}$ is the continuous move space:

$$\mathcal{A}_{\text{scan}} \in \mathbb{R} \mid -1 \leq \mathcal{A} \leq 1$$

$$\mathcal{A}_{\text{move}} \in \mathbb{R} \mid -1 \leq \mathcal{A} \leq 1$$

Using the information above, we discretize the continuous action spaces into discrete actions as:

$$a_{\text{scan}} = \begin{cases} N & \text{if } |a| < 0.33 \\ S_D & \text{if } 0.33 \leq |a| < 0.66 \\ S_R & \text{if } a \geq 0.66 \\ S_L & \text{if } a \leq -0.66 \end{cases}$$

$$a_{\text{move}} = \begin{cases} N & \text{if } |a| < 0.2 \\ L & \text{if } a < -0.2 \\ R & \text{if } a > 0.2 \end{cases}$$

C. Observation Space

The observation consists of the position the UAV is at and a noisy imaging mask:

$$o \in \mathcal{O}, \quad o = (p, \tilde{m})$$

To account for the error introduced by the radar instrumentation onboard the UAV, we express the observation model Z as:

$$Z(o \mid s', a) = \mathbb{I}(p_o = p') \cdot \prod_{i=1}^{N_t} \mathbb{P}(\tilde{m}_i \mid m'_i)$$

Where we model the observation that a region has correctly been imaged as:

$$\mathbb{P}(\tilde{m}_i \mid m'_i) = \begin{cases} 0.80 & \text{if } \tilde{m}_i = m'_i \\ 0.20 & \text{if } \tilde{m}_i \neq m'_i \end{cases}$$

D. Transition Function

As the error for the instrumentation is already accounted for in the observation function, the transition function defined below ensures that the drone will be able to accomplish the following:

- a) if a drone wants to move from one space to another, it will be able to do so absolutely
- b) if a drone scans a given area with a subsurface target, we can be certain it has scanned it

As such, the transition model can be expressed as:

$$T(s' \mid s, a) = \mathbb{I}[p' = f_p(p, a)] \cdot \mathbb{P}(m' \mid m, p, a)$$

Since movement is guaranteed, we can express p' as:

$$p' = \begin{cases} p - 1 & \text{if } a = L \\ p & \text{if } a = N \\ p + 1 & \text{if } a = R \end{cases}$$

Furthermore, the imaging state itself is represented as:

$$m'_i = \begin{cases} 1 & \text{if target } i \text{ is imaged} \\ m_i & \text{otherwise} \end{cases}$$

E. Reward Function and Discount Factor

When first designing our reward function, we wanted to ensure that the drone would quickly scan all subsurface targets. This led to the initial reward function design:

$$R(s, a, s') = \begin{cases} +100 & \text{if terminal} \\ -1 & \text{otherwise} \end{cases}$$

In running our code, however, we found that we needed to significantly tune our reward structure to prevent the trained policy from attempting to image over conductive regions. This led to:

$$R(s, a, s') = \begin{cases} +20 & \text{if terminal} \\ +5 & \text{if new target acquired} \\ -2 & \text{if old target imaged} \\ -0.25 & \text{if } a = N \\ -0.05 & \text{if } a = L, R \\ -0.1 & \text{otherwise} \end{cases}$$

Finally, we use a discount factor less than 1 in conjunction with the reward function above to encourage the UAV to collect the targets faster.

$$\gamma = 0.99$$

IV. Q-LEARNING IMPLEMENTATION

To implement Q-learning, we first must simplify the POMDP model down to an MDP model. In this assessment, we simply assume that the observed-image mask, $\tilde{m}$, is accurate to what is in the real world. In practice, the UAV and the solver would not know the conductive locations and target locations ahead of time. However, the optimal policy learned for the MDP could be used as a baseline / lower-bound policy for solving the POMDP. The Q-learning implementation Algorithm 1 used for this problem was adapted from [12] and can be found below.

Algorithm 1 Q-learning (Textbook / Class Notes Method)

```

1:  $Q(s, a) \leftarrow 0$ 
2:  $s \leftarrow s_0$ 
3: for each episode do
4:    $a \leftarrow \operatorname{argmax}_a Q(s, a)$  w.p.  $1 - \epsilon$ ,  $\operatorname{rand}(A)$  o.w.
5:    $r \leftarrow \operatorname{act}!(\operatorname{env}, a)$ 
6:    $s' \leftarrow \operatorname{observe}(\operatorname{env})$ 
7:    $Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$ 
8:    $s \leftarrow s'$ 
9: end for

```

The Q-learning algorithm above was tested with a 20 cell world. 8 subsurface targets and 8 conductive regions were chosen and hard-coded at [1, 5, 6, 7, 9, 15, 18, 19] and [2, 5, 6, 10, 13, 14, 15, 18], respectively. For the given scenario, we expect that the UAV would image down ($a = S_D$) at cell 1 if it starts there, as there is a target directly under it. Furthermore, if the drone is to start at cell 5, it should move to cell 4 ($a = L$) and then scan to the right ($a = S_R$). In our Q-learning implementation, we used a learning curve coefficient α and an epsilon coefficient ϵ of 0.1. For this first scenario, we completed 2000 episodes where the drone could move a maximum of 100 steps. Plotting and statistics were provided using existing Julia libraries (Plots and Statistics).

As seen in Figure 2, the UAV achieved a mean episode reward of 55.55 shortly before 500 episodes was reached. We found that the UAV learned to perform how we would expect it to at any given space with our reward function.

In addition to the 20 cell world with hard-coded target positions, we also completed a 100 cell world test case where 20 targets and 20 regions of conductivity were randomly placed. In this scenario, we completed 10000 episodes where the drone could move a maximum of 1000 steps.

As seen in Figure 2, the UAV achieved a mean episode reward of 95.62 before 2500 episodes was reached. We found that the UAV again learned the optimal policy that was expected given a cell world set up similar to that in Figure 1. However, as the world was larger, we needed more episodes for the learning curve to flatten.

Learning Curve: Agent Performance Over Training

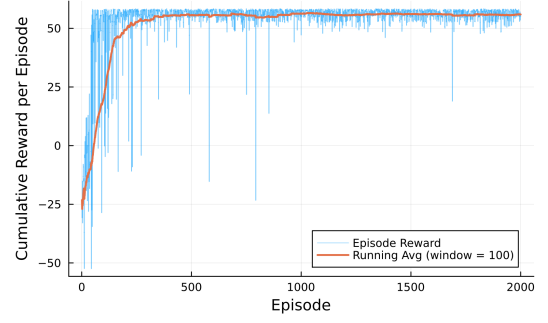


Fig. 2. Learning curve for Q-learning in simplified MDP model of UAV target extraction problem

Learning Curve: Agent Performance Over Training

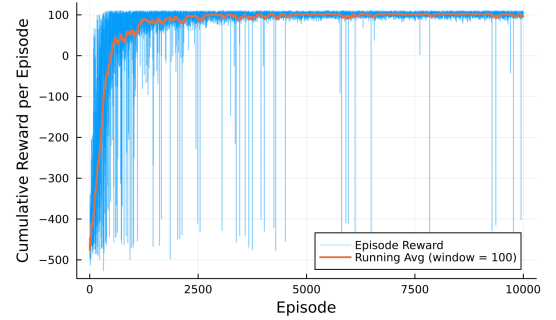


Fig. 3. Learning curve for Q-learning in larger but still simplified MDP model of UAV target extraction problem

V. SAC IMPLEMENTATION

We implemented SAC [13] in Python using the Stable-Baselines3 reinforcement learning library [14] with a standard multilayer perceptron policy. The custom environment was implemented in Gymnasium [15] with a continuous 2D action space enabling direct compatibility with SAC's continuous control formulation. The agent was trained off-policy using an experience replay buffer and mini-batch updates (size of 256) with our listed discount factor and a learning rate of 0.0003.

To support training evaluation, we implemented a custom callback which logged cumulative episodic rewards during learning. Environment compliance with the Gymnasium API was verified before training. The SAC model was trained for a varying number of timesteps, and performance was analyzed using reward learning curves and rollout visualization generated via Matplotlib.

We implemented SAC and assessed the performance in 3 stages, starting with a small cell world with static targets and ending with a large world with random target locations. As this is a POMDP, the solver needed to learn the conductive locations and target locations over time.

A. 6 Cell World, Static Targets

We first tested our POMDP with the SAC solver in a very simple 1D world composed of 6 cells with static targets

and conductive regions. We placed the conductive regions at $[2, 5]$ and the targets at $[1, 5]$. Therefore, we know that an optimal solution would be to image straight down at location 1, then move right to location 4 and image to the right to avoid the conductive patch at location 5. With our updated reward structure, we trained over a total of 25000 steps in the environment, where each episode in the environment could have a maximum of 100 steps.

Using our Python implementation of SAC, we achieved the learning curve shown in Figure 4. The agent achieved an average episode reward of 23.00.

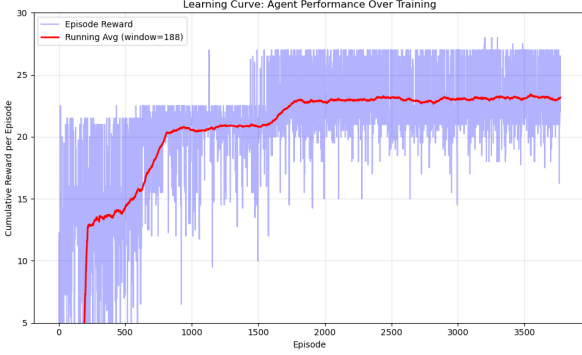


Fig. 4. Learning curve for Soft Actor Critic in the small world with static targets.

Below in Figure 5, we demonstrate how a single episode performs. This episode performed nearly perfectly and was able to handle the observation error at the end.

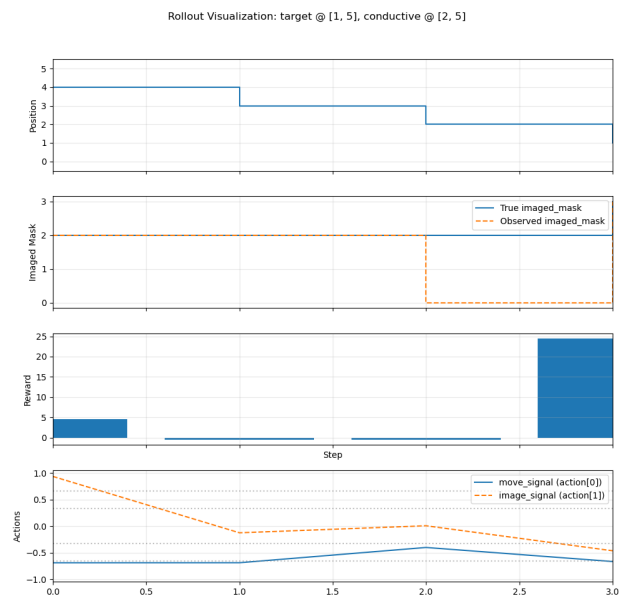


Fig. 5. Single episode of the optimal policy reached for the small static world.

B. 20 Cell World, Static Targets

The success of the SAC method within the 6 cell world indicated we should increase the complexity of the problem. To do this, we developed a 20 cell world with targets and conductive regions at $[1, 5, 6, 7, 9, 15, 18, 19]$ and $[2, 5, 6, 10, 13, 14, 15, 18]$, respectively. Additionally, to address the increasing complexity of the problem, we increased the total time steps over training in the environment up to 50,000 while keeping the max steps in a single episode at 100.

The learning curve from the 20 cell environment is shown below in Figure 6. The more complex environment led to significant variations in the learning rate, as the random placement of the drone in the environment made much more significant impact on the final reward than it did previously. Specifically, we see a peak average cumulative reward of near 45, approximately 10 less than the MDP formulation shown in Figure 2; this is likely a result of the observation error.

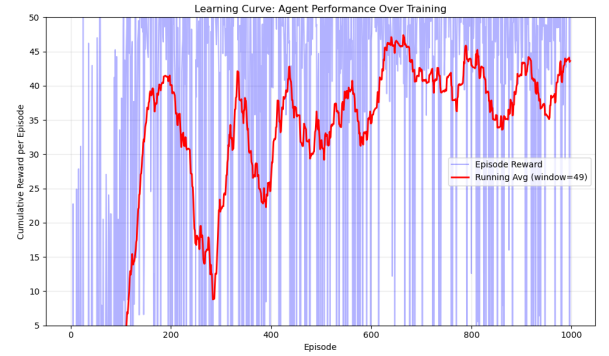


Fig. 6. Learning curve for the world with 20 cells and static targets and conductivity regions.

Again, a nearly ideal episode is shown in Figure 7. This episode demonstrates how the SAC method trained to find a near-optimal policy which knows the shortest path to successfully image all targets while avoiding the conductive patches. The policy is not completely optimal, as it attempted to image directly down over a conductive patch—resulting in a negative reward. However, the policy successfully images all targets efficiently, demonstrating that the SAC algorithm learned to solve the problem.

C. 20 Cell World, Randomized Target and Conductive Region Position

To implement randomized target and conductive region positions, a small modification was needed from the previously hard-coded values. Our code randomly chose 8 target positions and 8 conductive cell positions independent of one another. All POMDP parameters stayed the same (e.g., limiting a single episode to only 100 steps), and the only modification was an increase to 100,000 steps in the environment.

After training, we produced the learning curve shown in Figure 8. Compared to before, we see a much more unstable learning curve. This is expected, since so many state parameters are randomized that the reward from an individual episode

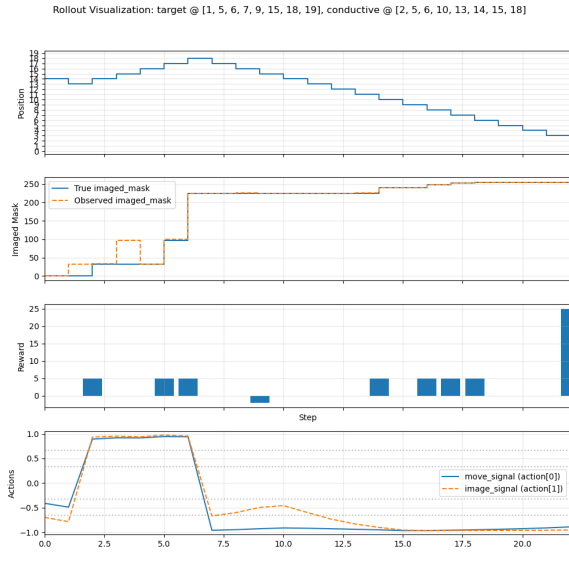


Fig. 7. A single, near-optimal episode for the 20-cell environment.

is highly dependent on the random starting position, target locations, and conductive regions. However, the algorithm still learned and converged to what we perceived as a near-optimal policy. We note that the cumulative reward per episode which SAC converges to is substantially less than what is shown in Figure 6. However, we expect this behavior. Since the target position changes, and the target and conductive locations are not known a priori, the drone needs to probe many positions to identify if there is a target or not. This oftentimes results in many negative rewards in search of identification of all targets to obtain the large terminal reward.

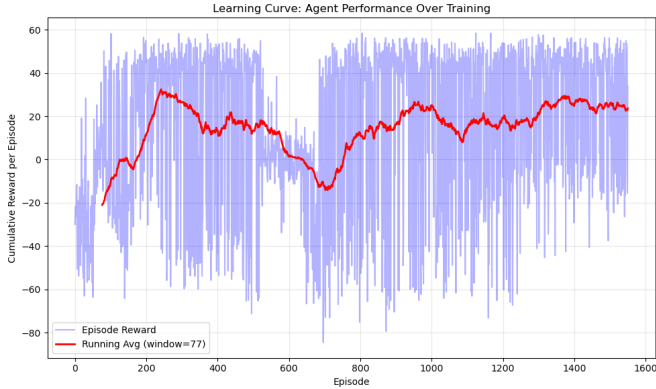


Fig. 8. Learning curve for SAC in POMDP with 8 randomized targets and conductive positions.

Figure 9 shows the behavior of the agent in a single episode. This episode shows a near-optimal performance—finding all targets and only attempting to image a false conductive region once even with significant observation errors. This is especially impressive when considering that the drone had no sense

of the target or conductive region locations. In this episode specifically, the drone started at the far end of the domain and worked its way towards the other side, imaging the subsurface along the way to avoid missing any potential targets. We can see that it optimized this approach as it learned to image in the same direction it was moving. This means that if there was a target in the direction of movement, the agent would get the reward from imaging it sooner, rather than having to waste an extra movement step and be penalized for such. This impressive performance highlights one of many successful episodes from the learned policy.

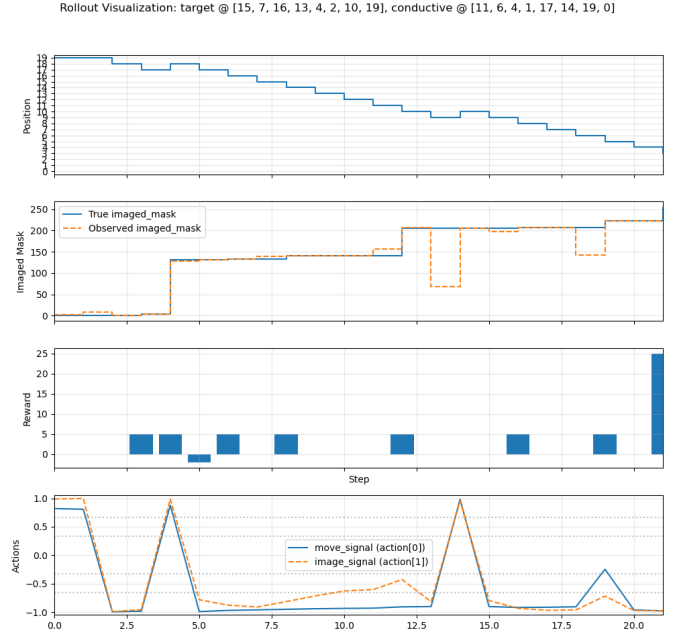


Fig. 9. A single episode of a near-optimal policy produced by SAC for the randomized 20 cell scenario.

VI. CONCLUSION

We demonstrated a novel application of Q-learning and Soft Actor Critic to solve a UAV subsurface imaging problem. We formulated a simple 1D scenario where the agent (i.e., the UAV) attempts to image subsurface targets that provide a positive reward. Above the targets, a heterogeneous conductive layer prevents direct imaging, which the drone can look around to see the subsurface. Our initial implementation as a simple MDP was successful. We were able to expand the MDP problem from a 20 cell world with preplanned targets to a 100 cell world with randomized targets. The policy that the agent took in the MDP formulation through Q-learning matched what we expected. Therefore, we advanced to a more realistic scenario with observation uncertainty for whether a target was successfully imaged. Starting with static target and conductive positions and a randomized starting point, SAC produced a near-optimal policy over a small domain of 6 cells. We then expanded to 20 cells with 8 targets and conductive areas. With

all else kept constant, SAC converged to a near-optimal policy with additional training; however, the learning curve was less stable due to the final reward having a greater dependence on the starting position. Finally, we modified the POMDP setup to have random target and conductivity positions for each episode. After 100,000 steps in the environment (about 1550 episodes), the output policy achieved a high average cumulative reward per episode, demonstrating that it almost always manages to image all subsurface targets while only getting a negative reward occasionally.

Overall, this demonstrates an outstanding performance of SAC for this application. The next logical step would be to apply this to a 2D position space, moving the problem formulation closer to reality with an ideal ending being the input of some prior information on the subsurface conductivity to get an optimal policy/path for the drone. To further move the problem towards a more realistic scenario, we should attempt to solve this problem with some form of an online POMCP algorithm (e.g., the POUCT algorithm discussed earlier). This algorithm and the optimal policies presented are analogs for many other real-world scenarios in remote sensing, including capture of LiDAR data, vegetation, radioglaciology, SAR, and more.

REFERENCES

- [1] T.-Z. Xiang, G.-S. Xia, and L. Zhang, "Mini-unmanned aerial vehicle-based remote sensing: Techniques, applications, and prospects," *IEEE geoscience and remote sensing magazine*, vol. 7, no. 3, pp. 29–63, 2019.
- [2] E. Schreiber, A. Heinzl, M. Peichl, M. Engel, and W. Wiesbeck, "Advanced buried object detection by multichannel, UAV/drone carried synthetic aperture radar," in *2019 13th European Conference on Antennas and Propagation (EuCAP)*. IEEE, 2019, pp. 1–5.
- [3] M. Frid and V. Frid, "Vital views into drone-based GPR application: precise mapping of soil-to-rock boundaries and ground water level for foundation engineering and site-specific response," *Applied Sciences*, vol. 14, no. 17, p. 7889, 2024.
- [4] J. Stasik and T. Szandala, "An autonomous drone for avalanche search and rescue: Integrating ArduPilot and Tracking-Beacon Detection," *IEEE Access*, 2025.
- [5] S. M. Tulaczyk and N. T. Foley, "The role of electrical conductivity in radar wave reflection from glacier beds," *The Cryosphere*, vol. 14, no. 12, pp. 4495–4506, 2020.
- [6] L. J. West, K. Handley, Y. Huang, and M. Pokar, "Radar frequency dielectric dispersion in sandstone: Implications for determination of moisture and clay content," *Water Resources Research*, vol. 39, no. 2, 2003.
- [7] K. Zheng, Y. Sung, G. Konidakis, and S. Tellex, "Multi-resolution POMDP planning for multi-object search in 3D," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 2022–2029.
- [8] D. Silver and J. Veness, "Monte-Carlo planning in large POMDPs," *Advances in neural information processing systems*, vol. 23, 2010.
- [9] J. Pajarinen and V. Kyrki, "Robotic manipulation of multiple objects as a POMDP," *Artificial Intelligence*, vol. 247, pp. 213–228, 2017.
- [10] S. A. Miller, Z. A. Harris, and E. K. Chong, "A POMDP framework for coordinated guidance of autonomous UAVs for multitarget tracking," *EURASIP Journal on Advances in Signal Processing*, vol. 2009, no. 1, p. 724597, 2009.
- [11] S. C. Ong, S. W. Png, D. Hsu, and W. S. Lee, "POMDPs for robotic tasks with mixed observability," in *Robotics: Science and systems*, vol. 5, no. 4, 2009.
- [12] M. J. Kochenderfer, T. A. Wheeler, and K. H. Wray, *Algorithms for decision making*. MIT press, 2022.
- [13] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel *et al.*, "Soft actor-critic algorithms and applications," *arXiv preprint arXiv:1812.05905*, 2018.
- [14] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of machine learning research*, vol. 22, no. 268, pp. 1–8, 2021.
- [15] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG *et al.*, "Gymnasium: A Standard Interface for Reinforcement Learning Environments," *arXiv preprint arXiv:2407.17032*, 2024.

CONTRIBUTIONS AND RELEASE

Both Duncan Byrne and Jonathan Williams contributed to the writing and formulation of this project. Each individual contributed to the creation of this paper, with Duncan Byrne leading the SAC implementation and conclusion sections, Jonathan Williams leading the introduction, related works, and Q-learning sections, and each contributing to the abstract and problem formulation sections.

Duncan Byrne utilized Python's StableBaselines3 implementation of Soft Actor Critic for all POMDP scenarios. The POMDP structure was implemented through the Gymnasium package. Standard Python libraries including numpy and matplotlib were used for result evaluation.

Jonathan Williams implemented the Q-learning algorithm in this project using the layout provided in the DMU Class textbook and Dr. Sunberg's class notes (Algorithm 1). Julia libraries (Plots, Random, and Statistics) were used to generate random targets and conductive regions and to generate plots for the project.

The authors grant permission for this work to be posted publicly.